3 哈希表
3.1 理论基础
3.1.1 哈希表
3.1.2 哈希函数
3.1.3 哈希碰撞
3.1.4 常见的三种哈希结构
3.2 经典题目
3.2.1 不排序查询
例题：leetcode 242 - 有效的字母异位词
例题：leetcode 349 - 两个数组的交集
3.2.2 循环排查
例题：leetcode 202 - 快乐数
3.2.3 哈希映射
例题：leetcode 1 - 两数之和
例题：leetcode 454 - 四数之和II
例题：leetcode 383 - 赎金信
3.2.4 滑动多指针
例题：leetcode 15 - 三数之和
例题：leetcode 18 - 四数之和

3 哈希表

3.1 理论基础

3.1.1 哈希表

- 哈希表（英文名字为Hash table，国内也有一些算法书籍翻译为散列表，看到这两个名称知道都是指hash table就可以了）是根据关键码的值而直接进行访问的数据结构。
    - 其实数组就是一张哈希表：哈希表中关键码就是数组的索引下标，然后通过下标直接访问数组中的元素。
- 哈希表能解决什么问题呢，**一般哈希表都是用来快速判断一个元素是否出现集合里。**
    - 例如要查询一个名字是否在这所学校里。要枚举的话时间复杂度是O(n)，但如果使用哈希表的话，只需要O(1)就可以做到。

3.1.2 哈希函数

- 就上述例子，哈希函数通过hashCode把名字转化为数值，一般hashcode是通过特定编码方式，可以将其他数据格式转化为不同的数值，这样就把学生名字映射为哈希表上的索引数字了。
- 如果hashCode得到的数值大于哈希表的大小了，也就是大于tableSize了，怎么办呢?
    - 此时为了保证映射出来的索引数值都落在哈希表上，我们会在再次对数值做一个取模的操作，这样我们就保证了学生姓名一定可以映射到哈希表上了。

3.1.3 哈希碰撞

- 在上述情形中，如果学生的数量大于哈希表的大小，此时就算哈希函数计算的再均匀，也避免不了会有几位学生的名字同时映射到哈希表同一个索引下标的位置，这种情况就叫做哈希碰撞。

- 一般哈希碰撞有两种解决方法，拉链法和线性探测法。
    - 拉链法：将发生冲突的元素都存储在链表中，这样我们就可以通过索引找到所有的元素。
    - 线性探测法：一定要保证tableSize大于dataSize。 我们需要依靠哈希表中的空位来解决碰撞问题。
        - 例如冲突的位置，放了小李，那么就向下找一个空位放置小王的信息。所以要求tableSize一定要大于dataSize ，要不然哈希表上就没有空置的位置来存放冲突的数据了。

3.1.4 常见的三种哈希结构

- 当我们想使用哈希法来解决问题的时候，我们一般会选择如下三种数据结构。
    - 数组
    - set（集合）
    - map(映射)
- 在C++中，set 和 map 分别提供以下三种数据结构，其底层实现以及优劣如下表所示：

集合	底层实现	是否有序	数值是否可以重复	能否更改数值	查询效率	增删效率
std::set	红黑树	有序	否	否	O(log n)	O(log n)
std::multiset	红黑树	有序	是	否	O(logn)	O(logn)
std::unordered_set	哈希表	无序	否	否	O(1)	O(1)

- std::unordered_set底层实现为哈希表，std::set 和std::multiset 的底层实现是红黑树，红黑树是一种平衡二叉搜索树，所以key值是有序的，但key不可以修改，改动key值会导致整棵树的错乱，所以只能删除和增加。
- **当我们要使用集合来解决哈希问题的时候，优先使用unordered_set，因为它的查询和增删效率是最优的，如果需要集合是有序的，那么就用set，如果要求不仅有序还要有重复数据的话，那么就用multiset。**

映射	底层实现	是否有序	数值是否可以重复	能否更改数值	查询效率	增删效率
std::map	红黑树	key有序	key不可重复	key不可修改	O(logn)	O(logn)
std::multimap	红黑树	key有序	key可重复	key不可修改	O(log n)	O(log n)
std::unordered_map	哈希表	key无序	key不可重复	key不可修改	O(1)	O(1)

- std::unordered_map 底层实现为哈希表，std::map 和std::multimap 的底层实现是红黑树。同理，std::map 和std::multimap 的key也是有序的（这个问题也经常作为面试题，考察对语言容器底层的理解）。
- 总结一下，**当我们遇到了要快速判断一个元素是否出现集合里的时候，就要考虑哈希法**。但是哈希法也是**牺牲了空间换取了时间**，因为我们要使用额外的数组，set或者是map来存放数据，才能实现快速的查找。
    - 如果在做面试题目的时候遇到需要判断一个元素是否出现过的场景也应该第一时间想到哈希法！

3.2 经典题目

3.2.1 不排序查询

例题：leetcode 242 - 有效的字母异位词

[242. 有效的字母异位词]

- 给定两个字符串 `s` 和 `t` ，编写一个函数来判断 `t` 是否是 `s` 的 字母异位词。

```
class Solution {
public:
    bool isAnagram(string s, string t) {
        int r[26]={0};
        if(s.length()!=t.length()) return false;
        for(int i=0;i<s.length();i++){
            r[s[i]-'a']++;
            r[t[i]-'a']--;
        }
        for(auto &i:r) if(i!=0) return false;
        return true;
    }
};
```

- 因为不需要考虑字母的位置所以我们就单纯地造一个record的数组，用哈希表来计数对比即可。

例题：leetcode 349 - 两个数组的交集

[349. 两个数组的交集]

给定两个数组 `nums1` 和 `nums2` ，返回 它们的交集 。输出结果中的每个元素一定是 **唯一** 的。我们可以 **不考虑输出结果的顺序** 。

```
class Solution {
public:
    vector<int> intersection(vector<int>& nums1, vector<int>& nums2) {
        int r1[1001]={0},r2[1001]={0};
        vector<int> res;
        for(int i=0;i<nums1.size();i++) r1[nums1[i]]++;
        for(int i=0;i<nums2.size();i++) r2[nums2[i]]++;
        for(int i=0;i<1001;i++) if(r1[i]!=0&&r2[i]!=0) res.push_back(i);
        return res;
    }
};
```

- 类似于上道题的思路，我们也是构建了一个record数组也能通过，但注意到：
    - 如果哈希值比较少、特别分散、跨度非常大，使用数组就造成空间的极大浪费！

```
class Solution {
public:
    vector<int> intersection(vector<int>& nums1, vector<int>& nums2) {
        unordered_set<int> result_set;
        unordered_set<int> nums_set(nums1.begin(), nums1.end());
        for (int num : nums2) {
            if (nums_set.find(num) != nums_set.end()) {
                result_set.insert(num);
            }
        }
        return vector<int>(result_set.begin(), result_set.end());
    }
};
```

- 通过上述我们可以学到哈希表的创建、查找、插入和转换成vector的基本逻辑。

3.2.2 循环排查

例题：leetcode 202 - 快乐数

[202. 快乐数]

- 编写一个算法来判断一个数 `n` 是不是快乐数。
- **「快乐数」** 定义为：
    - 对于一个正整数，每一次将该数替换为它每个位置上的数字的平方和。
    - 然后重复这个过程直到这个数变为 1，也可能是 **无限循环** 但始终变不到 1。
    - 如果这个过程 **结果为** 1，那么这个数就是快乐数。
- 如果 `n` 是 快乐数 就返回 `true` ；不是，则返回 `false` 。

```
class Solution {
public:
    int op(int n){
        int sum = 0;
        while(n!=0){
            sum+=(n%10)*(n%10);
            n = (n-n%10)/10;
        }
        return sum;
    }

    bool isHappy(int n) {
        unordered_set<int> re;
        while(!re.count(n)){
            if(n==1) return true;
            re.insert(n);
            n = op(n);
        }
        return false;
    }
```

```
};
```

- 对于这种题目的一个难点就是排查死循环是否出现：
  - 什么时候会出现呢？就是当年在操作时发现某一个数已经在之前出现过了，例如：2，4，16，37，58，89，145，42，20，**4**，注意到当你检测到第二次出现4的时候就可以下结论了，它永远也到不了1。
- 所以对于上述情况，我们公式化地采取哈希表的方法就可以解决：
  - 构建一个哈希表记录流程：
    - 检查这个数出现过没有：有则立刻停止返回false；
    - 没出现过就插入这个数，并进入下一个循环。

3.2.3 哈希映射

例题：leetcode 1 - 两数之和

[1. 两数之和]

- 给定一个整数数组 `nums` 和一个整数目标值 `target`，请你在该数组中找出 **和为目标值** *`target`* 的那 **两个** 整数，并返回它们的数组下标。
- 你可以假设每种输入只会对应一个答案，并且你不能使用两次相同的元素。
- 你可以按任意顺序返回答案。

```
class Solution {
public:
    vector<int> twoSum(vector<int>& nums, int target) {
        unordered_map<int,int> mp;
        for(int i=0;i<nums.size();i++){
            if(mp.find(nums[i])==mp.end()) mp[nums[i]]=1;
            else mp[nums[i]]++;
        }

        vector<int> res(2);
        for(int i=0;i<nums.size();i++){
            mp[nums[i]]--;
            if(!mp[nums[i]]) mp.erase(nums[i]);
            if(mp.find(target-nums[i])!=mp.end()) res[0] = i;}

        for(int i=0;i<nums.size();i++) if(nums[i]==target - nums[res[0]]&&i!=res[0]) res[1]=i;

        return res;
    }
};
```

- 为什么不用哈希set呢，因为在这个题目中我们需要计数，（否则遇到target=两个一样的数相加的话就会产生一定的问题）。

例题：leetcode 454 - 四数之和II

454. 四数相加 II

给你四个整数数组 `nums1`、`nums2`、`nums3` 和 `nums4` ，数组长度都是 `n` ，请你计算有多少个元组 `(i, j, k, l)` 能满足：

- `0 <= i, j, k, l < n`
- `nums1[i] + nums2[j] + nums3[k] + nums4[l] == 0`

```
class Solution {
public:
    int fourSumCount(vector<int>& nums1, vector<int>& nums2, vector<int>& nums3, vector<int>& nums4) {
        int n = nums1.size(),sum=0;
        unordered_map<int,int> mp;
        for(auto &a:nums1) for(auto &b:nums2) mp[a+b]++;
        for(auto &c:nums3) for(auto &d:nums4) if(mp.find(-c-d)!=mp.end()) sum+=mp[-c-d];
        return sum;
    }
};
```

- 分而治之：把四个数组其实分成了两个部分处理，实现了O(n^2)的计算速度，同时也优化了哈希map的写法。

例题：leetcode 383 - 赎金信

383. 赎金信

- 给你两个字符串： `ransomNote` 和 `magazine` ，判断 `ransomNote` 能不能由 `magazine` 里面的字符构成。
- 如果可以，返回 `true` ；否则返回 `false` 。
- `magazine` 中的每个字符只能在 `ransomNote` 中使用一次。

```
class Solution {
public:
    bool canConstruct(string ransomNote, string magazine) {
        unordered_map<char,int> mp;
        for(auto &a:magazine) mp[a]++;

        for(auto &b:ransomNote){
            if(mp.find(b)==mp.end()) return false;
            if(!--mp[b]) mp.erase(b);
        }
        return true;
    }
};
```

- 这道题也是经典的需要统计出现过没有，并且要计数不需要顺序的题目：采用unordered_map。

3.2.4 滑动多指针

例题：leetcode 15 - 三数之和

15. 三数之和

- 给你一个整数数组 `nums` ，判断是否存在三元组 `[nums[i], nums[j], nums[k]]` 满足 `i != j`、`i != k` 且 `j != k` ，同时还满足 `nums[i] + nums[j] + nums[k] == 0` 。请你返回所有和为 `0` 且不重复的三元组。
- **注意：**答案中不可以包含重复的三元组。

```
class Solution {
public:
    vector<vector<int>> threeSum(vector<int>& nums) {
        int n = nums.size();
        vector<vector<int>> res;
        sort(nums.begin(),nums.end());
        for(int i=0;i<n-2;i++){
            if(nums[i]>0) break;
            if(i>0&&nums[i]==nums[i-1]) continue;
            int left = i+1,right = n-1;
            while(left<right){
                if(nums[i]+nums[left]+nums[right]==0){
                    res.push_back(vector<int>{nums[i],nums[left],nums[right]});
                    while(right>0&&nums[right-1]==nums[right]) right--;
                    while(left<n-1&&nums[left+1]==nums[left]) left++;
                    left++;right--;
                }
                else if(nums[i]+nums[left]+nums[right]<0) left++;
                else right--;
            }
        }
        return res;
    }
};
```

- 这道题如果采用哈希表首先统计前两个再算第三个会面临一个问题：不允许重复数值出现，这会导致这个题目很难通过这个方法AC。所以我们采用滑动三指针法。
    - 首先对数组排序（注意到就题目的算法而言，很难突破O(n^2)的速度，所以排序的代价是可以接受的）
    - 然后固定一个左端点，作为可能存在答案的第一个数，因为本身是排序后的，所以只需要滑动左断点时，把相同的数滑过去就可以避免重复。
    - 然后对于左端点和数组末尾区间的数用双指针找值即可。

例题：leetcode 18 - 四数之和

- 给你一个由 `n` 个整数组成的数组 `nums` ，和一个目标值 `target` 。请你找出并返回满足下述全部条件且**不重复**的四元组 `[nums[a], nums[b], nums[c], nums[d]]` （若两个四元组元素一一对应，则认为两个四元组重复）：
    - `0 <= a, b, c, d < n`

- `a`、`b`、`c` 和 `d` **互不相同**
- `nums[a] + nums[b] + nums[c] + nums[d] == target`

- 你可以按 **任意顺序** 返回答案 。

[18. 四数之和]

```
class Solution {
public:
    vector<vector<int>> fourSum(vector<int>& nums, int target) {
        int n = nums.size();
        vector<vector<int>> res;
        sort(nums.begin(),nums.end());

        for(int a=0;a<n-3;a++){
            if(a>0&&nums[a]==nums[a-1]) continue;

            for(int d=n-1;d>=a+3;d--){
                if(d<n-1&&nums[d]==nums[d+1]) continue;
                int b=a+1,c=d-1;

                while(b<c){
                    long long sum = (long long)nums[a]+nums[b]+nums[c]+nums[d];
                    if(sum==target){
                        res.push_back(vector<int>{nums[a],nums[b],nums[c],nums[d]});
                        while(b<c&&nums[c-1]==nums[c]) c--;
                        while(b<c&&nums[b+1]==nums[b]) b++;
                        b++;c--;
                    }
                    else if(sum<target) b++;
                    else c--;
                }

            }
        }
        return res;
    }
};
```

- 完全类似于三数之和，只需要在尾部多加一个指针就行了。